

Data Structures

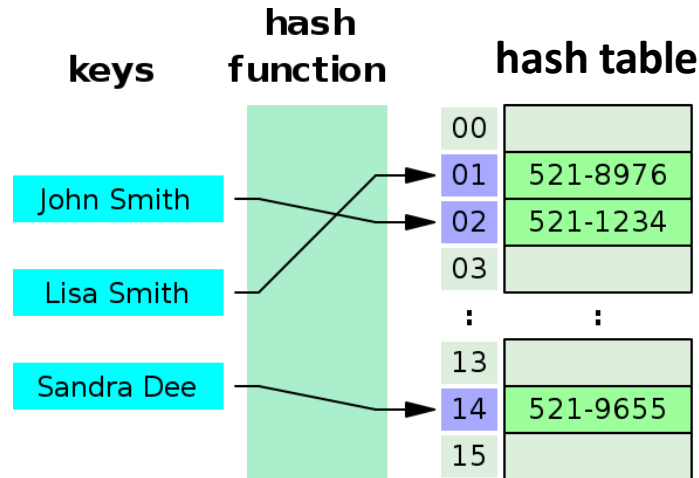
Lecture 19: Search – Hashing (2)

Soobin Um

May 20, 2026

Recap: Hashing (해싱)

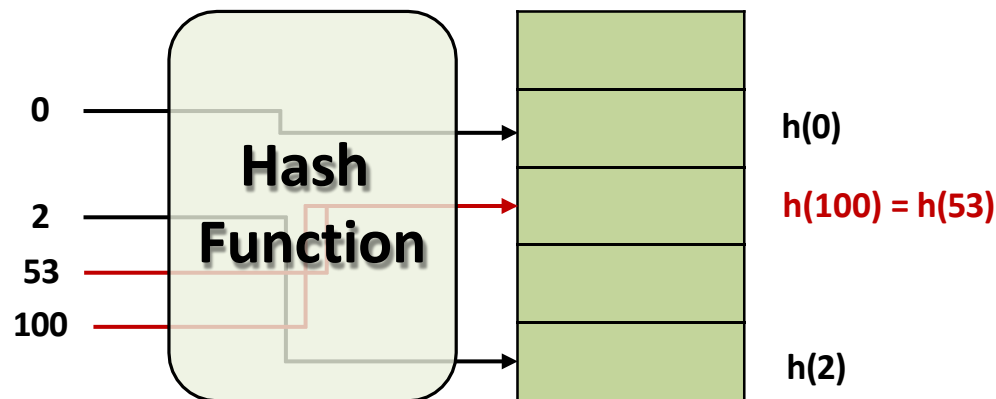
- 해시 함수 $h(k)$ 를 이용하여, 키 k 를 해시 테이블의 특정 인덱스 또는 슬롯으로 매핑
 - 해시 테이블 HT : 아이템을 저장하는 크기 m 의 배열
 - 해시 함수 $h(\cdot)$: 키 k 를 $0 \sim (m-1)$ 범위의 값으로 매핑
 - Ex) $h(k) = k \% m$
- 키가 k 인 아이템은 해시 함수를 통해 얻은 해시 테이블의 인덱스 위치 $HT[h(k)]$ 에 저장됨



https://en.wikipedia.org/wiki/Hash_table

Collision (충돌)

- 두 개 이상의 서로 다른 키가 동일한 해시 함수 값을 갖는 상황
 - 동일한 슬롯에 여러 아이템들이 몰리면서 탐색, 삽입, 삭제 등의 연산에 추가적인 작업 필요하게 됨
 - 그 결과, 해시 테이블의 **성능이 크게 저하**될 수 있음
 - 충돌이 많아질수록 성능 저하 심화
- 충돌을 줄이기 위한 기본 전략
 - **해시 함수를 잘 설계**함으로써, 충돌 발생 가능성을 최소화해야 함
 - 그럼에도, “**전체 키 개수 $n >$ 해시 테이블 크기 m** ”일 경우, **충돌은 반드시 발생**하게 됨 (비둘기 집 원리)
 - 따라서, 충돌 발생 시 이를 처리하기 위한 충돌 해결 방법(collision resolution)이 필수적



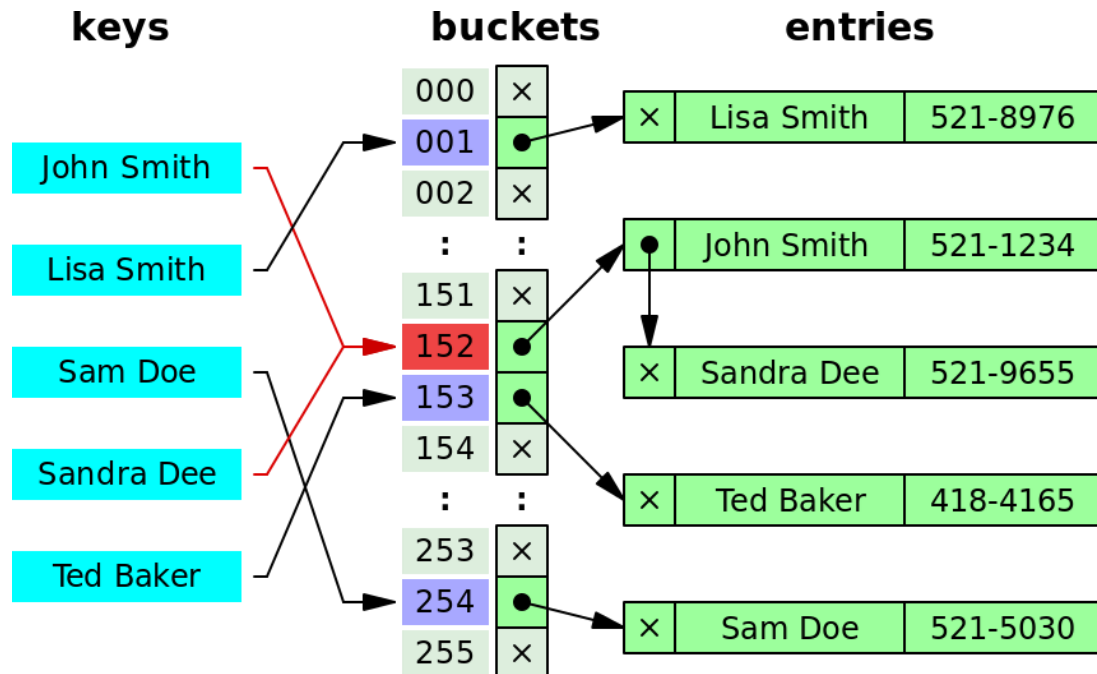
Collision Resolution

Collision Resolution (충돌 해결)

- 서로 다른 두 개 이상의 키가 동일한 해시 값을 갖는 경우, 충돌(collision)이 발생
- 충돌 발생 시 이를 처리하기 위한 전략은 크게 두 가지 방식으로 분류됨
 - **Open Hashing** (or Separate Chaining)
 - **Closed Hashing** (or Open Addressing)

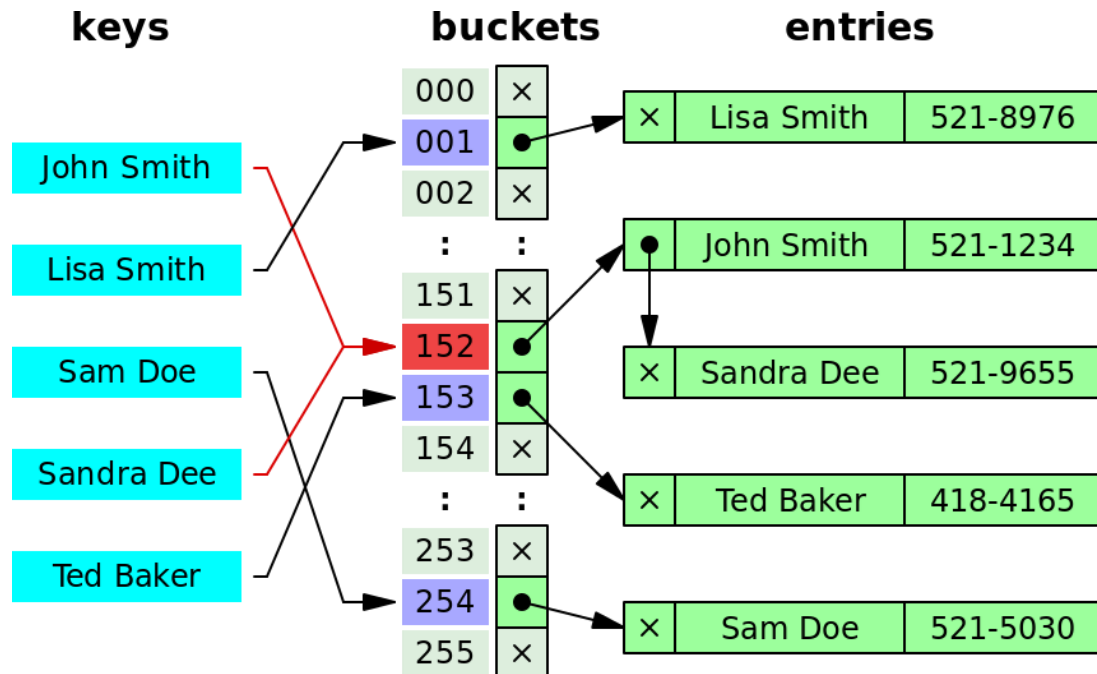
Open Hashing (Chaining)

- 해시 테이블의 각 슬롯에 연결 리스트(linked list)를 연결
- 삽입 시 충돌이 발생하면, 해당 슬롯의 연결 리스트에 추가
 - 즉, 슬롯 내부에 일종의 “체인”을 만들어 충돌을 처리



특징: Open Hashing

- 데이터 저장 공간이 해시 테이블 밖으로 확장
 - 테이블 밖의 연결 리스트에 계속해서 삽입 가능 → 테이블 크기에 덜 민감
- 연결 리스트의 노드만 제거하면 됨 → 삭제가 용이



Closed Hashing (Open Addressing)

- 모든 아이템이 해시 테이블 내부에만 저장됨
- 삽입 시 충돌이 발생하면, 해시 테이블의 다른 빈 슬롯을 탐색(probing)하여 저장

Closed Hashing (Open Addressing)

- 모든 아이템이 **해시 테이블 내부에만** 저장됨
- 삽입 시 충돌이 발생하면, 해시 테이블의 **다른 빈 슬롯을 탐색(probing)**하여 저장

특징

- 추가 메모리 구조 불필요 (메모리 사용 효율이 높음)
- 해시 테이블이 많이 채워져 있을수록 (load factor가 높을수록) 성능이 빠르게 악화됨
- 삭제 시 해당 슬롯에 대한 별도의 “마킹”이 필요

Open vs. Closed Hashing

	Open Hashing	Closed Hashing
저장 위치	해시 테이블 + 외부 구조	해시 테이블 내부
장점	• Load factor가 높을 때도 안정적 • 삭제가 용이	• 메모리 효율이 좋음 • 캐시 적중률 높음 (→ 실행 속도 빠름)
단점	• 추가 메모리 공간 필요	• Load factor 높을 시, 성능 하락 • 삭제 연산 복잡
대표 기법	Linked list chaining	Bucket hashing, linear probing

Bucket Hashing

- 개념: 해시 테이블을 여러 개의 버킷(bucket)으로 그룹화
 - 전체 m 개의 슬롯을 b 개의 버킷으로 분할
 - 각 버킷은 m/b 개의 슬롯으로 구성

	Hash Table	Overflow
0	1000 9530	1057
1		
2	9877 2007	
3	3013	
4	9879	

$m = 10, b = 5$

Bucket Hashing

- **개념:** 해시 테이블을 여러 개의 버킷(bucket)으로 그룹화
 - 전체 m 개의 슬롯을 b 개의 버킷으로 분할
 - 각 버킷은 m/b 개의 슬롯으로 구성
- **해시 함수**
 - 해시 함수 $h(k)$ 는 버킷의 번호만 결정
 - 아이템은 해당 버킷의 첫 번째 슬롯에 삽입됨

Hash Table		Overflow
0	1000 9530	1057
1		
2	9877 2007	
3	3013	
4	9879	

$$m = 10, b = 5, h(k) = k \bmod b$$

Bucket Hashing

- **개념:** 해시 테이블을 여러 개의 버킷(bucket)으로 그룹화
 - 전체 m 개의 슬롯을 b 개의 버킷으로 분할
 - 각 버킷은 m/b 개의 슬롯으로 구성
- **해시 함수**
 - 해시 함수 $h(k)$ 는 버킷의 번호만 결정
 - 아이템은 해당 버킷의 첫 번째 슬롯에 삽입됨
- **충돌 처리**
 - 슬롯이 차 있을 경우, 같은 버킷의 다음 슬롯으로 삽입
 - 버킷이 모두 가득 차 있을 경우, overflow bucket에 삽입
 - Overflow bucket: 추가 메모리 공간

	Hash Table	Overflow
0	1000 9530	1057
1		
2	9877 2007	
3	3013	
4	9879	

$$m = 10, b = 5, h(k) = k \bmod b$$

키 삽입 순서: 9877, 2007, 1000, 9530, 3013, 9879, 1057

Variant of Bucket Hashing

- **개념:** 버킷이 없는 것처럼 해싱 수행
 - 해시 함수가 슬롯을 직접 지정
 - 단, 실제 저장은 해당 슬롯이 속한 버킷 단위로 처리
 - 버킷 나뉘기는 유지하면서, 더 고른 분포를 갖도록 함

	Hash Table	Overflow
0	1000	1057
1	9530	
2		
3	3013	
4		
5		
6	2007	
7	9877	
8		
9	9879	

$$m = 10, b = 5, h(k) = k \bmod m$$

Variant of Bucket Hashing

- **개념:** 버킷이 없는 것처럼 해싱 수행
 - 해시 함수가 슬롯을 직접 지정
 - 단, 실제 저장은 해당 슬롯이 속한 버킷 단위로 처리
 - 버킷 나뉘기는 유지하면서, 더 고른 분포를 갖도록 함
- **충돌 처리**
 - 슬롯이 비어 있을 경우, 바로 삽입
 - 슬롯이 차 있을 경우, **같은 버킷 내 다른 빈 슬롯 탐색** 후 삽입
 - 즉, 버킷 내부에서만 탐색 수행
 - 버킷의 모든 슬롯이 차 있으면, overflow bucket에 삽입

	Hash Table	Overflow
0	1000	1057
1	9530	
2		
3	3013	
4		
5		
6	2007	
7	9877	
8		
9	9879	

$$m = 10, b = 5, h(k) = k \bmod m$$

키 삽입 순서: 9877, 2007, 1000, 9530, 3013, 9879, 1057

Limitation

- 낮은 해시 테이블 공간 활용도
 - 해시 테이블에 빈 공간이 많이 남아 있음에도, 버킷이 가득 차 있을 경우 overflow bucket에 아이터를 넣게 됨

Hash Table	Overflow	Hash Table	Overflow
0 1000	1057	0 1000	1057
9530		1 9530	
1		2	
2 9877		3 3013	
2007		4	
3 3013		5	
		6 2007	
4 9879		7 9877	
		8	
		9 9879	

Linear Probing (선형 탐색)

- 개념: 버킷 없이 closed hashing을 수행
 - 충돌 발생 시, 해시 테이블 내에서 빈 슬롯을 직접 탐색(probe)하여 삽입
 - 별도의 버킷 구조 이용하지 않음

Linear Probing (선형 탐색)

- **개념:** 버킷 없이 closed hashing을 수행
 - 충돌 발생 시, 해시 테이블 내에서 **빈 슬롯을 직접 탐색(probe)**하여 삽입
 - 별도의 버킷 구조 이용하지 않음
- **충돌 처리(probe sequence)**
 - 빈 슬롯 검사

$$h(k) + p(k, i)$$

$$\text{where } p(k, i) = i$$

- 초기 위치: $h(k)$ (i : 충돌 발생 횟수)
 - 첫 번째 충돌 발생 시 $\rightarrow h(k) + 1$
 - 두 번째 충돌 발생 시 $\rightarrow h(k) + 2$
 - 세 번째 충돌 발생 시 $\rightarrow h(k) + 3$
 - ...

예제: Linear Probing

$$h(k) + p(k, i)$$

where $p(k, i) = i$

0	9050
1	1001
2	
3	
4	
5	
6	
7	9877
8	2037
9	

$m = 10$, $h(k) = k \bmod m$ (a)

키 삽입 순서: 1001, 9050, 9877, 2037

예제: Linear Probing

$$h(k) + p(k, i)$$

where $p(k, i) = i$

0	9050
1	1001
2	
3	
4	
5	
6	
7	9877
8	2037
9	

(a)

0	9050
1	1001
2	
3	
4	
5	
6	
7	9877
8	2037
9	1059

(b)

$m = 10$, $h(k) = k \bmod m$

키 삽입 순서: 1001, 9050, 9877, 2037

1059 추가 삽입

Limitation

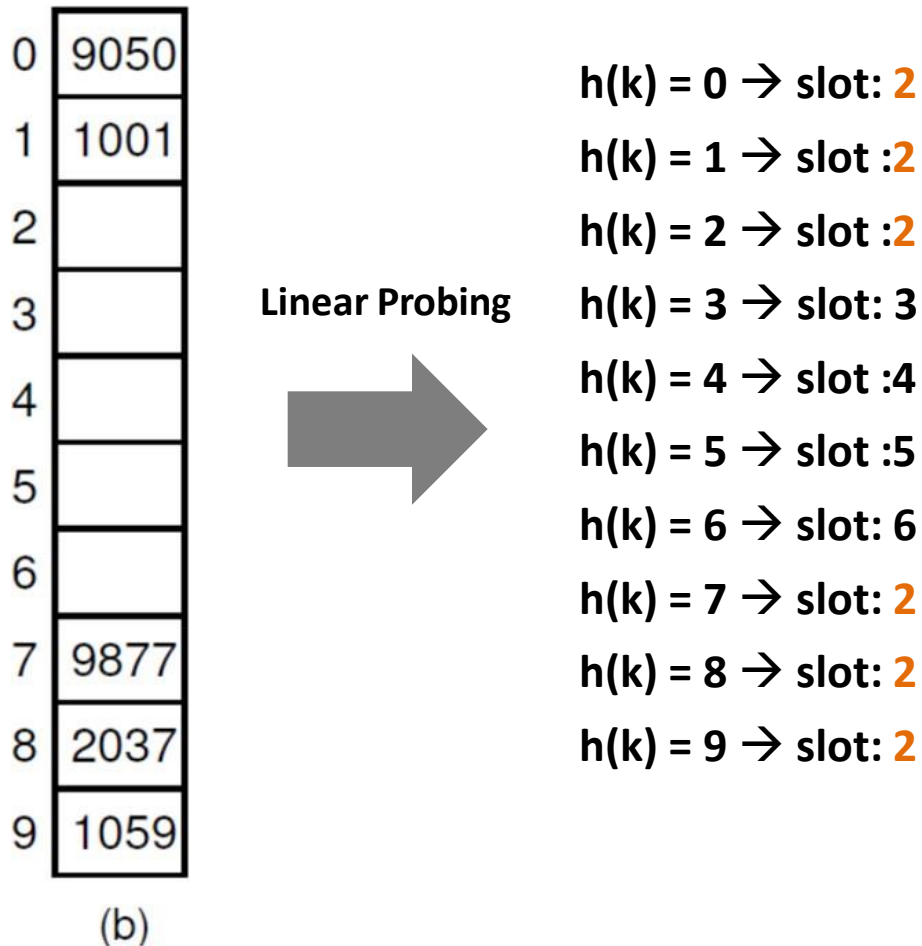
- 채워져 있는 슬롯들이 점점 뭉치게 되는 “primary clustering” 발생
- 남은 슬롯들에 할당될 확률이 슬롯마다 서로 달라지게 됨

0	9050
1	1001
2	
3	
4	
5	
6	
7	9877
8	2037
9	1059

(b)

Limitation

- 채워져 있는 슬롯들이 점점 뭉치게 되는 “primary clustering” 발생
- 남은 슬롯들에 할당될 확률이 슬롯마다 서로 달라지게 됨



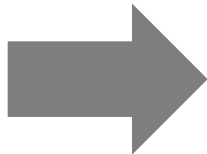
Limitation

- 채워져 있는 슬롯들이 점점 뭉치게 되는 “**primary clustering**” 발생
- 남은 슬롯들에 할당될 확률이 슬롯마다 서로 달라지게 됨

0	9050
1	1001
2	
3	
4	
5	
6	
7	9877
8	2037
9	1059

(b)

Linear Probing



$h(k) = 0 \rightarrow \text{slot: } 2$

$h(k) = 1 \rightarrow \text{slot: } 2$

$h(k) = 2 \rightarrow \text{slot: } 2$

$h(k) = 3 \rightarrow \text{slot: } 3$

$h(k) = 4 \rightarrow \text{slot: } 4$

$h(k) = 5 \rightarrow \text{slot: } 5$

$h(k) = 6 \rightarrow \text{slot: } 6$

$h(k) = 7 \rightarrow \text{slot: } 2$

$h(k) = 8 \rightarrow \text{slot: } 2$

$h(k) = 9 \rightarrow \text{slot: } 2$

다음 삽입 시 해시 함수에 의해
각 슬롯이 선택될 확률

slot 2: 6/10

slot 3: 1/10

slot 4: 1/10

slot 5: 1/10

slot 6: 1/10

즉, 채워진 구간 뒤의 첫 번째 빈 슬롯에 해시 값들이 집중되면서, 빈 슬롯 간의 할당 확률 달라짐

Improved Linear Probing

- 상수배 건너뛰기

$$h(k) + p(k, i)$$

$$\text{where } p(k, i) = c \cdot i$$

- 단, c 는 m 과 서로소여야 함
- 서로소가 아닐 경우, probe 시 특정 슬롯들만 반복적으로 방문하게 됨
 - $m=6, c=2$: $h(0)$ 에서 충돌 발생 시 이후 탐색은 $h(2), h(4), h(0), h(2), h(4) \dots$

Improved Linear Probing

- 상수배 건너뛰기

$$h(k) + p(k, i)$$

$$\text{where } p(k, i) = c \cdot i$$

- 단, c 는 m 과 서로소여야 함
- 서로소가 아닐 경우, probe 시 특정 슬롯들만 반복적으로 방문하게 됨
 - $m=6, c=2$: $h(0)$ 에서 충돌 발생 시 이후 탐색은 $h(2), h(4), h(0), h(2), h(4) \dots$
- Primary clustering 문제 여전히 존재
- Probe 간격만 바뀔 뿐, 연속된 클러스터는 그대로 유지

Improved Linear Probing

- **Pseudo-random probing**

$$h(k) + p(k, i)$$

$$\text{where } p(k, i) = \text{Perm}[i - 1]$$

- 임의의 (랜덤) 순서로 슬롯을 probe
 - Perm: 1부터 m-1 사이의 값을 무작위로 섞은 길이 m-1의 배열

Improved Linear Probing

- Pseudo-random probing

$$h(k) + p(k, i)$$

$$\text{where } p(k, i) = \text{Perm}[i - 1]$$

- 임의의 (랜덤) 순서로 슬롯을 probe
 - Perm: 1부터 m-1 사이의 값을 무작위로 섞은 길이 m-1의 배열
- Probe 경로가 슬롯마다 다르므로 primary clustering 완화
- 하지만 캐시 친화성(적중률)이 낮아질 수 있음

Improved Linear Probing

- Quadratic probing

$$h(k) + p(k, i)$$

$$\text{where } p(k, i) = c_1 i^2 + c_2 i + c_3$$

- Probe 간격이 점점 벌어지므로, primary clustering 문제 감소
- 하지만 secondary clustering 문제가 여전히 남아 있음

※ Secondary Clustering

같은 해시 값을 갖는 키들이 같은 probe 경로를 사용하기 때문에 발생하는 문제
(즉, 클러스터의 원인이 probe 방식에 있는 것이 아니라, 같은 해시 값을 갖는 키 집단에 있음)

Improved Linear Probing

- Double hashing

$$h(k) + p(k, i)$$

$$\text{where } p(k, i) = i \cdot h_2(k)$$

- 두 번째 해시 함수(h_2)를 이용하여 probe step을 키마다 다르게 설정
 - 첫 번째 충돌 시 $\rightarrow h(k) + h_2(k)$
 - 두 번째 충돌 시 $\rightarrow h(k) + 2h_2(k)$
 - 세 번째 충돌 시 $\rightarrow h(k) + 3h_2(k)$

Improved Linear Probing

- Double hashing

$$h(k) + p(k, i)$$

$$\text{where } p(k, i) = i \cdot h_2(k)$$

- 두 번째 해시 함수(h_2)를 이용하여 probe step을 키마다 다르게 설정
 - 첫 번째 충돌 시 $\rightarrow h(k) + h_2(k)$
 - 두 번째 충돌 시 $\rightarrow h(k) + 2h_2(k)$
 - 세 번째 충돌 시 $\rightarrow h(k) + 3h_2(k)$
- 같은 해시 값 즉, $h(k)$ 를 갖는 키라도, $h_2(k)$ 가 다르면 probe sequence 자체가 달라짐
 - Secondary clustering 문제 해소
- Pseudo-random, quadratic probing 기법과 혼용 가능

Analysis of Closed Hashing

- **Load Factor (적재율, α)**
 - $\alpha = s/m$ (s: 저장된 아이템 수, m: HT 사이즈)
 - $\alpha = 0.7 \rightarrow$ HT의 70%가 차 있음

Analysis of Closed Hashing

- **Load Factor (적재율, α)**

- $\alpha = s/m$ (s: 저장된 아이템 수, m: HT 사이즈)
 - $\alpha = 0.7 \rightarrow$ HT의 70%가 차 있음

- **아이템을 HT에 넣는데 걸리는 평균 탐색 횟수**

- 첫 번째 탐색에서 성공할 확률 $P(1)$ (빈 슬롯을 바로 만날 확률):

$$P(1) = 1 - \frac{s}{m}$$

Analysis of Closed Hashing

- **Load Factor (적재율, α)**

- $\alpha = s/m$ (s : 저장된 아이템 수, m : HT 사이즈)
 - $\alpha = 0.7 \rightarrow$ HT의 70%가 차 있음

- **아이템을 HT에 넣는데 걸리는 평균 탐색 횟수**

- 첫 번째 탐색에서 성공할 확률 $P(1)$ (빈 슬롯을 바로 만날 확률):

$$P(1) = 1 - \frac{s}{m}$$

- 두 번째 탐색에서 성공할 확률 $P(2)$:

$$P(2) = \frac{s}{m} \left(1 - \frac{s-1}{m-1} \right)$$

Analysis of Closed Hashing

- **Load Factor (적재율, α)**

- $\alpha = s/m$ (s : 저장된 아이템 수, m : HT 사이즈)
 - $\alpha = 0.7 \rightarrow$ HT의 70%가 차 있음

- **아이템을 HT에 넣는데 걸리는 평균 탐색 횟수**

- 첫 번째 탐색에서 성공할 확률 $P(1)$ (빈 슬롯을 바로 만날 확률):

$$P(1) = 1 - \frac{s}{m}$$

- 두 번째 탐색에서 성공할 확률 $P(2)$:

$$P(2) = \frac{s}{m} \left(1 - \frac{s-1}{m-1} \right)$$

- i 번째 탐색에서 성공할 확률 $P(i)$:

$$P(i) = \frac{s(s-1) \cdots (s-i+2)}{m(m-1) \cdots (m-i+2)} \left(1 - \frac{s-i+1}{m-i+1} \right)$$

Analysis of Closed Hashing

- 아이템을 HT에 넣는데 걸리는 평균 탐색 횟수

- 1번째 탐색에서 성공할 확률 $P(i)$:

$$P(i) = \frac{s(s-1)\cdots(s-i+2)}{m(m-1)\cdots(m-i+2)} \left(1 - \frac{s-i+1}{m-i+1}\right)$$

- 평균 탐색 횟수

$$\sum_{i=1}^{s+1} i \cdot P(i)$$

Analysis of Closed Hashing

- 아이템을 HT에 넣는데 걸리는 평균 탐색 횟수

- 1번째 탐색에서 성공할 확률 $P(i)$:

$$P(i) = \frac{s(s-1)\cdots(s-i+2)}{m(m-1)\cdots(m-i+2)} \left(1 - \frac{s-i+1}{m-i+1}\right)$$

- 평균 탐색 횟수

$$\sum_{i=1}^{s+1} i \cdot P(i) = 1 + \frac{s}{m} + \frac{s(s-1)}{m(m-1)} + \cdots$$

Analysis of Closed Hashing

- 아이템을 HT에 넣는데 걸리는 평균 탐색 횟수

- 1번째 탐색에서 성공할 확률 $P(i)$:

$$P(i) = \frac{s(s-1)\cdots(s-i+2)}{m(m-1)\cdots(m-i+2)} \left(1 - \frac{s-i+1}{m-i+1}\right)$$

- 평균 탐색 횟수

Assume: $m \gg s$ 충분히 큼

$$\sum_{i=1}^{s+1} i \cdot P(i) = 1 + \frac{s}{m} + \frac{s(s-1)}{m(m-1)} + \cdots \approx 1 + \sum_{i=1}^{\infty} \left(\frac{s}{m}\right)^i$$

Analysis of Closed Hashing

- 아이템을 HT에 넣는데 걸리는 평균 탐색 횟수

- 1번째 탐색에서 성공할 확률 $P(i)$:

$$P(i) = \frac{s(s-1)\cdots(s-i+2)}{m(m-1)\cdots(m-i+2)} \left(1 - \frac{s-i+1}{m-i+1}\right)$$

- 평균 탐색 횟수

$$\sum_{i=1}^{s+1} i \cdot P(i) = 1 + \frac{s}{m} + \frac{s(s-1)}{m(m-1)} + \cdots \approx 1 + \sum_{i=1}^{\infty} \left(\frac{s}{m}\right)^i = \frac{1}{1-\alpha}$$

Analysis of Closed Hashing

- **아이템을 HT에 넣는데 걸리는 평균 탐색 횟수**
 - 1번째 탐색에서 성공할 확률 $P(i)$:

$$P(i) = \frac{s(s-1)\cdots(s-i+2)}{m(m-1)\cdots(m-i+2)} \left(1 - \frac{s-i+1}{m-i+1}\right)$$

- **평균 탐색 횟수**

$$\sum_{i=1}^{s+1} i \cdot P(i) = 1 + \frac{s}{m} + \frac{s(s-1)}{m(m-1)} + \cdots \approx 1 + \sum_{i=1}^{\infty} \left(\frac{s}{m}\right)^i = \frac{1}{1-\alpha}$$

Closed hashing 평균 탐색 횟수는 적재율(α)에 크게 좌우됨

Look ahead

Graph